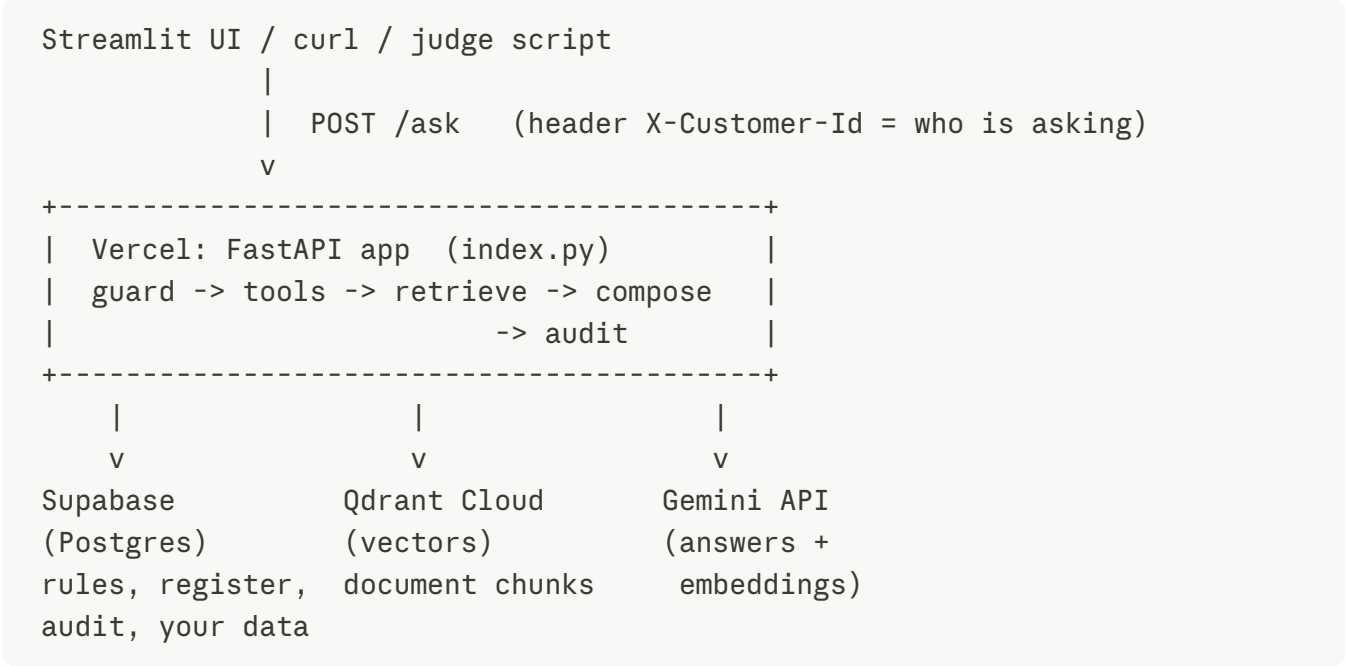
HCL RAG Agent — Project Guide

Oct 29, 2026 · @Vydik

The agent answers policy and account questions only from your own documents and data, cites every answer, and refuses when it cannot back an answer up. It runs as a FastAPI app on Vercel, keeps tables in Supabase, document chunks in Qdrant, and uses Gemini to write answers.

At a glance

A question goes in at the API on Vercel; code checks who is asking, pulls facts from Supabase and Qdrant, and Gemini only phrases the answer.



Where things live

Four hosted services hold everything; the code itself keeps no state, so any Vercel instance can serve any request.

Service	What it does here	Dashboard	Settings it needs
Vercel	Runs the FastAPI app (`index.py` → `app/main.py`)	vercel.com/dashboard	All variables in Settings, added under Project → Settings → Environment Variables

Service	What it does here	Dashboard	Settings it needs
Supabase (project ctrl_freaks, eu-west-1)	Postgres tables: Source Register, rule registry, audit log, your domain tables	supabase.com/dashboard	`DATABASE_URL`
Qdrant Cloud	Vector search over document chunks (collection `docs`)	cloud.qdrant.io	`QDRANT_URL`, `QDRANT_API_KEY`
Google AI Studio (Gemini)	Writes answers (`gemini-flash-latest`) and embeds text (`gemini-embedding-001`, 3072 dims)	ai.studio/projects	`GEMINI_API_KEY`

The Streamlit UI (`ui/app.py`) is not on Vercel. Run it on a laptop or Streamlit Cloud with `API_URL` pointing at the Vercel URL.

Settings (environment variables)

Locally they live in `hcl-rag-agent/.env` (never committed); on Vercel they go in Project → Settings → Environment Variables. Same names in both places.

Variable	What it does	Where to get it
`DATABASE_URL`	Supabase Postgres connection	Supabase → Connect → Direct → Transaction pooler (port 6543) → copy URI, put your database password in place of `[YOUR-PASSWORD]` (no brackets)
`QDRANT_URL`	Qdrant cluster address	Qdrant Cloud → cluster → endpoint URL (ends `:6333`)
`QDRANT_API_KEY`	Qdrant access	Qdrant Cloud → cluster → API Keys
`QDRANT_COLLECTION`	Collection name, default `docs`	Leave as is
`GEMINI_API_KEY`	Gemini chat + embeddings	Google AI Studio → Get API key (the project needs credits)

Variable	What it does	Where to get it
`LLM_MODEL`	Answer model, default `gemini-flash-latest`	Change only to switch model
`EMBED_MODEL`	Embedding model, default `gemini-embedding-001`	Changing it means deleting the Qdrant collection and re-ingesting
`EMBED_BACKEND`	`api` (Gemini) or `hash` (offline tests)	`api` in real use
`MOCK_LLM`	`true` skips Gemini for answers	`false` in real use
`DOMAIN`	Which domain plug-in runs (`generic`, `retail`, yours)	Set by `scripts/new_domain.py`
`SUBJECT_HEADER`	Header that names the user, default `X-Customer-Id`	Match the problem doc
`ADMIN_KEY`	Unlocks admin endpoints and any audit record	Make up a long random value; change from `change-me`
`TOP_K`	Chunks used per answer, default 6	Tune with the eval
`MIN_RELEVANCE`	Below this similarity the bot says not found	Raise to about 0.6–0.65 for Gemini, then tune with the eval
`CHUNK_CHARS`, `CHUNK_OVERLAP`	Chunk size 900, overlap 150 characters	Rarely changed
`TEST_DATABASE_URL`	Throwaway Postgres for tests (tables get emptied)	Local only, never on Vercel, never the real database

API endpoints

Base URL is your Vercel address (`https://<your-app>.vercel.app`) or `http://localhost:8000` locally. Interactive docs with a Try-it button are at `/docs`.

Method + path	What it does	Who can call it
`POST /ask`	Ask a question; returns a cited answer	Anyone; send `X-Customer-Id` to ask about your own records

Method + path	What it does	Who can call it
`POST /ingest`	Upload one document (PDF, DOCX, MD, TXT) plus its Source Register details; live at once, no restart	Anyone (put it behind admin if the problem doc requires)
`GET /health`	Status of API, database, vector store and Gemini	Anyone
`GET /audit/{trace_id}`	Full record of how one answer was made	The user who asked (same header) or admin (`X-Admin-Key`); others get 404
`GET /sources`	All registered documents	Anyone
`GET /rules`	All rules (thresholds) with their source document	Anyone
`POST /admin/rules?source_doc_id=DOC`	Add or replace one rule	Admin (`X-Admin-Key`)
`POST /admin/load-data`	Upload data CSVs, one `<table>.csv` per table	Admin (`X-Admin-Key`)

Ask a question:

```
curl -X POST https://<your-app>.vercel.app/ask \
  -H "Content-Type: application/json" -H "X-Customer-Id: C1001" \
  -d '{"question": "Can I return the headphones from order O-100001?",
"as_of_date": "2026-10-06"}'
```

The reply always has the same fields: `trace_id` , `answer` , `answer_type` (one of retrieved_fact, calculated, not_found, clarification_needed, refused, conflict_flagged), `citations` , `tools_invoked` , `applied_rules` , `conflicts_detected` , `upcoming_changes` , `assumptions` , `explanation` , `as_of_date` . These names are the judged contract in `app/schemas.py` .

Upload a document:

```
curl -X POST https://<your-app>.vercel.app/ingest -F file=@circular.pdf -F
'metadata={
  "doc_id": "CIRC-2026-10", "title": "Window change", "authority_level": 2,
  "effective_from": "2026-10-01", "supersedes": "CIRC-2026-07#1"}'
```

`authority_level` runs 1 (highest) to 5 (unofficial, never sets a rule). Add `"rules": [...]` to register thresholds the document sets.

How a question gets answered

The flow is a fixed LangGraph pipeline in `app/agent.py`; code makes every decision and calculation, Gemini only phrases.

1. **Guard** — reads who is asking from the header only. Asking about another person's id, or a record that is not theirs, is refused with the same message either way. Injection-like wording is flagged.
2. **Tools** (personal questions) — the domain's Python tools read the user's own rows from Supabase and apply thresholds from the rule registry, e.g. return window days. Every value used is recorded and cited.
3. **Retrieve** — hybrid search: Qdrant vector search plus BM25 keyword search, merged by rank (RRF), top 6 kept. The precedence filter then drops chunks that are superseded, not yet in force or out of scope.
4. **Compose** — Gemini writes the answer from the numbered evidence only. Too-weak evidence gives `not_found`; sources that disagree and cannot be ranked give `conflict_flagged`.
5. **Audit** — one record per answer in Supabase: sources, tools with inputs and outputs, rules, conflicts, model, tokens, latency, safety flags.

Which source wins when documents disagree: in force on the date asked and in scope → not superseded → lower authority level → later effective date → more specific scope → otherwise flagged as a conflict with both cited.

Guardrails

The main guardrail is design: Gemini never decides or calculates, and every Gemini call has a non-LLM fallback.

Guardrail	What it stops	Where
Identity from header only	Reading someone else's records by typing their id	`app/agent.py` (guard)
Same refusal for "not yours" and "does not exist"	Guessing which ids exist	`app/agent.py`, `/audit`
Evidence cleaning	Instructions hidden in documents steering the answer	`app/guard.py`

Guardrail	What it stops	Where
Flagged questions skip Gemini	Prompt injection through the question	`app/agent.py` (compose)
Number check on answers and explanations	Gemini inventing a number not in the evidence or tool output	`app/agent.py`
Closed intent list, numeric-only rule extraction	Gemini making up intents or rules	`app/domain/base.py`, `app/ingest/pipeline.py`
Relevance threshold	Answering from weak matches	`MIN_RELEVANCE`
Redacted audit log	Emails, phones, card numbers stored in logs	`app/guard.py`

Limits: the injection check is English keyword matching and can be paraphrased around; there is no harmful-content filter.

Where to change what

For a new problem statement you normally touch only your domain file and data folder.

I want to…	Change this
Start a new problem statement	Run `python scripts/new_domain.py <name>`, then fill `app/domain/<name>.py`
Add tables, intents, tools, categories	`app/domain/<name>.py` (copy patterns from `examples/retail/domain.py`)
Add documents and thresholds	`data/<name>/docs/`, `source_register.csv`, `csv/rule_registry.csv`
Add eval questions	`data/<name>/eval_set.jsonl`
Rename API fields to match the problem doc	`app/schemas.py`, `app/main.py`
Switch Gemini model or provider	`LLM_MODEL`, `LLM_BASE_URL` (any OpenAI-compatible API)
Change search behaviour	`TOP_K`, `MIN_RELEVANCE`; code in `app/retrieval/store.py`

I want to…	Change this
Change database access	`app/db.py` (write SQL with `?` placeholders; it converts them for Postgres)
Change UI labels	`ui/app.py`

`app/agent.py`, `app/precedence.py`, `app/rules.py`, `app/retrieval/`, `app/ingest/` and `app/domain/base.py` are core: change them only with the integrator's OK.

Run it locally and load data

1. `pip install -r requirements-dev.txt` (use a virtual environment).
2. Copy `.env.example` to `.env` and fill in the Settings above.
3. `python scripts/load_data.py` — loads the active domain's documents into Qdrant and its tables and rules into Supabase. Add `--reingest` to re-index documents already loaded.
4. `uvicorn app.main:app --reload` — API at `http://localhost:8000/docs`.
5. `streamlit run ui/app.py` with `API_URL` set to the API address.

Loading data from a laptop is enough for the live site: Vercel reads the same Supabase and Qdrant.

Tests and eval: start a throwaway Postgres (`docker compose up -d db`), then `pytest -q` and `python eval/run_eval.py --label <name>`. They empty the tables they use, so they must never point at the real Supabase.

Deploy and update on Vercel

1. Push `hcl-rag-agent` to a GitHub repo (`.env` stays out via `.gitignore`).
2. On vercel.com, Add New → Project → Import the repo. Set Root Directory to the `hcl-rag-agent` folder if it is not the repo root; leave the framework on auto-detect (it finds FastAPI through `index.py`).
3. Under Environment Variables, paste the whole `.env` into the first key box; Vercel splits it. Remove `TEST_DATABASE_URL`.
4. Deploy, then open `https://<your-app>.vercel.app/health`: database, vector_store and llm should all say ok.

To update: push to GitHub and Vercel redeploys. To change a setting: edit it in Settings → Environment Variables, then Redeploy from the Deployments tab.

Known limits and to-dos

- [ ] Tune `MIN_RELEVANCE` for Gemini embeddings with the eval (0.30 is too low; unrelated text scores about 0.55).
- [ ] Change `ADMIN_KEY` from its default, locally and on Vercel.
- [ ] Run `pytest -q` and the eval once a test Postgres is up; not yet run on the new stack.
- [ ] Decide on a reranker (Cohere/Jina API or Gemini) — none today.
- [ ] Confirm the hackathon allows Qdrant, Supabase and Gemini instead of the listed ChromaDB, SQLite and Ollama.

- Gemini needs prepaid credits; when they run out every call returns 402 and answers fall back to plain extracts.
- Changing `EMBED_MODEL` means deleting the Qdrant `docs` collection and re-ingesting.